

4.7 Lab: Logistic Regression, LDA, QDA, and KNN

4.7.1 The Stock Market Data

Trong lab này, ta sẽ khảo sát dữ liệu **Smarket**, một phần của thư viện **ISLP**. Dataset này gồm percentage return của chỉ số chứng khoán S&P 500 trong 1,250 ngày, từ đầu năm 2001 đến cuối năm 2005. Với mỗi ngày, ta ghi lại percentage return của từng ngày trong năm ngày giao dịch trước đó, từ **Lag1** đến **Lag5**. Ta cũng ghi lại **Volume** (số cổ phiếu được giao dịch trong ngày trước đó, tính theo tỷ), **Today** (percentage return vào ngày đang xét) và **Direction** (thị trường đi Up hay Down trong ngày đó).

Ta bắt đầu bằng cách import các thư viện ở top level; đây đều là những import đã gặp trong các lab trước.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                        summarize)
```

Ta cũng gom các import mới cần dùng cho lab này.

```
In [2]: from ISLP import confusion_table
from ISLP.models import contrast
from sklearn.discriminant_analysis import \
    (LinearDiscriminantAnalysis as LDA,
     QuadraticDiscriminantAnalysis as QDA)
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

Bây giờ ta đã sẵn sàng load dữ liệu **Smarket**.

```
In [3]: Smarket = load_data('Smarket')
Smarket
```

Lệnh này cho một listing dữ liệu đã bị rút gọn mà ta không hiển thị ở đây. Ta có thể xem tên các variable.

```
In [4]: Smarket.columns
```

```
Out[4]: Index(['Year', 'Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume',
              'Today', 'Direction'],
              dtype='object')
```

Ta tính correlation matrix bằng method **corr()** cho data frame; method này tạo một matrix chứa toàn bộ pairwise correlation giữa các variable. (Ta không hiển thị output ở đây.) Thư viện **pandas** không báo correlation cho variable **Direction** vì đây là một qualitative variable.

`.corr()`

In [5]: `Smarket.corr()`

Đúng như kỳ vọng, correlation giữa các lagged return variable và return của ngày hôm nay đều gần bằng 0. Correlation đáng kể duy nhất là giữa **Year** và **Volume**. Khi plot dữ liệu, ta thấy **Volume** tăng theo thời gian. Nói cách khác, số cổ phiếu trung bình được giao dịch mỗi ngày tăng từ năm 2001 đến 2005.

In [6]: `Smarket.plot(y='Volume');`

4.7.2 Logistic Regression

Tiếp theo, ta fit một logistic regression model để dự đoán **Direction** bằng **Lag1** đến **Lag5** và **Volume**. Function `sm.GLM()` fit generalized linear model, một class of models bao gồm logistic regression. Ngoài ra, function `sm.Logit()` fit trực tiếp một logistic regression model. Syntax của `sm.GLM()` tương tự `sm.OLS()`, ngoại trừ việc ta phải truyền argument `family=sm.families.Binomial()` để cho `statsmodels` biết cần chạy logistic regression thay vì một generalized linear model loại khác.

*sm.GLM()
generalized
linear model*

In [7]:

```
allvars = Smarket.columns.drop(['Today', 'Direction', 'Year'])
design = MS(allvars)
X = design.fit_transform(Smarket)
y = Smarket.Direction == 'Up'
glm = sm.GLM(y,
             X,
             family=sm.families.Binomial())
results = glm.fit()
summarize(results)
```

Out[7]:

	coef	std err	z	P> z
intercept	-0.1260	0.241	-0.523	0.601
Lag1	-0.0731	0.050	-1.457	0.145
Lag2	-0.0423	0.050	-0.845	0.398
Lag3	0.0111	0.050	0.222	0.824
Lag4	0.0094	0.050	0.187	0.851
Lag5	0.0103	0.050	0.208	0.835
Volume	0.1354	0.158	0.855	0.392

p -value nhỏ nhất ở đây gắn với **Lag1**. Coefficient âm của predictor này gợi ý rằng nếu thị trường có return dương vào hôm qua thì hôm nay khả năng thị trường tăng sẽ thấp hơn. Tuy nhiên, với giá trị 0.15, p -value vẫn tương đối lớn, vì vậy chưa có bằng chứng rõ ràng về một association thực sự giữa **Lag1** và **Direction**.

Ta dùng attribute `params` của `results` để truy cập chỉ các coefficient của fitted model này.

In [8]: `results.params`

Out[8]:

intercept	-0.126000
Lag1	-0.073074
Lag2	-0.042301
Lag3	0.011085
Lag4	0.009359
Lag5	0.010313

```
Volume      0.135441
dtype: float64
```

Tương tự, ta có thể dùng attribute `pvalues` để truy cập các p -value của coefficient (không hiển thị).

In [9]:

```
results.pvalues
```

Method `predict()` của `results` có thể được dùng để dự đoán probability rằng thị trường sẽ tăng, khi biết các giá trị của predictor. Method này trả về prediction trên probability scale. Nếu không cung cấp dataset cho function `predict()`, thì probability được tính cho training data đã dùng để fit logistic regression model. Tương tự linear regression, ta có thể truyền một optional argument `exog` phù hợp với một design matrix nếu muốn. Ở đây ta chỉ in mười probability đầu tiên.

In [10]:

```
probs = results.predict()
probs[:10]
```

Out[10]:

```
array([0.5070841, 0.4814679, 0.4811388, 0.5152223, 0.5107812,
       0.5069565, 0.4926509, 0.5092292, 0.5176135, 0.4888378])
```

Để tạo prediction về việc thị trường sẽ tăng hay giảm trong một ngày cụ thể, ta phải chuyển các predicted probability này thành class label, **Up** hoặc **Down**. Hai command sau tạo một vector class prediction dựa trên việc predicted probability của thị trường tăng lớn hơn hay nhỏ hơn 0.5.

In [11]:

```
labels = np.array(['Down']*1250)
labels[probs > 0.5] = "Up"
```

Function `confusion_table()` từ package ISLP tóm tắt các prediction này, cho biết bao nhiêu observation được classification đúng hoặc sai. Function của chúng ta, được adapted từ một function tương tự trong module `sklearn.metrics`, transpose matrix kết quả và thêm label cho hàng và cột. Function `confusion_table()` nhận argument thứ nhất là predicted label và argument thứ hai là true label.

`confusion_`
`table()`

In [12]:

```
confusion_table(labels, Smarket.Direction)
```

Out[12]:

	Truth	Down	Up
Predicted			
Down		145	141
Up		457	507

Các phần tử trên đường chéo của confusion matrix biểu thị prediction đúng, còn các phần tử ngoài đường chéo biểu thị prediction sai. Vì vậy model của ta dự đoán đúng rằng thị trường sẽ tăng trong 507 ngày và giảm trong 145 ngày, tổng cộng $507 + 145 = 652$ prediction đúng. Function `np.mean()` có thể được dùng để tính tỷ lệ ngày mà prediction là đúng. Trong trường hợp này, logistic regression dự đoán đúng chuyển động của thị trường 52.2% số lần.

In [13]: `(507+145)/1250, np.mean(labels == Smarket.Direction)`

Out[13]: (0.5216, 0.5216)

Thoạt nhìn, có vẻ logistic regression model hoạt động tốt hơn random guessing một chút. Tuy nhiên, kết quả này gây hiểu nhầm vì ta train và test model trên cùng một tập gồm 1,250 observation. Nói cách khác, $100 - 52.2 = 47.8\%$ là training error rate. Như đã thấy trước đây, training error rate thường quá lạc quan — nó có xu hướng đánh giá thấp test error rate. Để đánh giá tốt hơn accuracy của logistic regression model trong setting này, ta có thể fit model bằng một phần dữ liệu rồi xem nó dự đoán held-out data tốt đến đâu. Cách này sẽ cho một error rate thực tế hơn, theo nghĩa rằng trong thực tế ta quan tâm đến performance của model không phải trên dữ liệu đã dùng để fit model, mà trên những ngày tương lai mà chuyển động thị trường còn chưa biết.

Để triển khai strategy này, trước tiên ta tạo một Boolean vector tương ứng với các observation từ năm 2001 đến 2004. Sau đó dùng vector này để tạo một held-out dataset gồm các observation của năm 2005.

In [14]:

```
train = (Smarket.Year < 2005)
Smarket_train = Smarket.loc[train]
Smarket_test = Smarket.loc[~train]
Smarket_test.shape
```

Out[14]: (252, 9)

Object `train` là một vector gồm 1,250 phần tử, tương ứng với các observation trong dataset. Các phần tử của vector tương ứng với observation xảy ra trước năm 2005 được đặt thành `True`, còn những phần tử tương ứng với observation trong năm 2005 được đặt thành `False`. Do đó `train` là một boolean array, vì các phần tử của nó là `True` và `False`. Boolean array có thể được dùng để lấy subset các row hoặc column của data frame bằng method `loc`.

Chẳng hạn, command `Smarket.loc[train]` sẽ chọn ra một submatrix của dataset thị trường chứng khoán chỉ tương ứng với các ngày trước năm 2005, vì đó là những ngày mà phần tử của `train` bằng `True`. Symbol `~` có thể được dùng để negate toàn bộ phần tử của một Boolean vector. Tức là, `~train` là một vector giống `train`, ngoại trừ các phần tử `True` trong `train` được đổi thành `False` trong `~train`, và ngược lại. Vì vậy, `Smarket.loc[~train]` cho một subset các row của data frame thị trường chứng khoán chỉ chứa những observation mà `train` bằng `False`. Output ở trên cho biết có 252 observation như vậy.

Bây giờ ta fit một logistic regression model chỉ bằng subset các observation tương ứng với các ngày trước năm 2005. Sau đó ta thu được predicted probability rằng thị trường chứng khoán sẽ tăng cho từng ngày trong test set — tức các ngày của năm 2005.

In [15]:

```
X_train, X_test = X.loc[train], X.loc[~train]
y_train, y_test = y.loc[train], y.loc[~train]
glm_train = sm.GLM(y_train,
                    X_train,
```



```
results = glm_train.fit()
probs = results.predict(exog=X_test)
labels = np.array(['Down']*252)
labels[probs > 0.5] = 'Up'
confusion_table(labels, L_test)
```

Out[19]:

	Truth	Down	Up
Predicted			
Down		35	35
Up		76	106

Hãy đánh giá overall accuracy cũng như accuracy trong những ngày mà logistic regression dự đoán thị trường tăng.

In [20]:

```
(35+106)/252, 106/(106+76)
```

Out[20]:

```
(0.5595, 0.5824)
```

Bây giờ kết quả có vẻ tốt hơn một chút: 56% chuyển động hằng ngày được dự đoán đúng. Đáng chú ý là trong trường hợp này, một strategy đơn giản hơn rất nhiều — dự đoán thị trường sẽ tăng mỗi ngày — cũng đúng 56% số lần! Vì vậy, xét về overall error rate, method logistic regression không tốt hơn naive approach. Tuy nhiên, confusion matrix cho thấy vào những ngày logistic regression dự đoán thị trường tăng, nó đạt accuracy 58%. Điều này gợi ý một trading strategy khả dĩ: mua vào những ngày model dự đoán thị trường tăng và tránh giao dịch vào những ngày dự đoán giảm. Dĩ nhiên ta cần khảo sát cẩn thận hơn xem cải thiện nhỏ này là thực hay chỉ do random chance.

Giả sử ta muốn dự đoán return ứng với các giá trị cụ thể của **Lag1** và **Lag2**. Cụ thể, ta muốn dự đoán **Direction** vào một ngày mà **Lag1** và **Lag2** lần lượt bằng 1.2 và 1.1, và một ngày mà chúng bằng 1.5 và -0.8 . Ta thực hiện việc này bằng function `predict()`.

In [21]:

```
newdata = pd.DataFrame({'Lag1': [1.2, 1.5],
                        'Lag2': [1.1, -0.8]});
newX = model.transform(newdata)
results.predict(newX)
```

Out[21]:

```
0    0.4791
1    0.4961
dtype: float64
```

4.7.3 Linear Discriminant Analysis

Ta bắt đầu bằng cách thực hiện LDA trên dữ liệu **Smarket**, dùng function `LinearDiscriminantAnalysis()`, mà ta viết tắt là `LDA()`. Ta fit model chỉ bằng các observation trước năm 2005.

In [22]:

```
lda = LDA(store_covariance=True)
```

*Linear
Discriminant
Analysis()*

Vì LDA estimator tự động thêm intercept, ta nên loại column tương ứng với intercept trong cả **X_train** lẫn **X_test**. Ta cũng có thể dùng trực tiếp các label thay vì các Boolean vector **y_train**.

```
In [23]: X_train, X_test = [M.drop(columns=['intercept'])
                        for M in [X_train, X_test]]
          lda.fit(X_train, L_train)
```

```
Out[23]: LinearDiscriminantAnalysis(store_covariance=True)
```

Ở đây ta đã dùng list comprehension được giới thiệu trong Mục 3.6.4. Nhìn vào dòng đầu tiên phía trên, ta thấy về phải là một list có length bằng hai. Lý do là code `for M in [X_train, X_test]` iterate qua một list có length bằng hai. Dù ở đây ta loop qua một list, method list comprehension hoạt động khi loop qua bất kỳ iterable object nào. Sau đó ta áp dụng method `drop()` cho từng phần tử trong iteration và thu thập kết quả vào một list. Về trái bảo Python unpack list có length bằng hai này, gán các phần tử của nó cho variable `X_train` và `X_test`. Dĩ nhiên điều này overwrite các giá trị trước đó của `X_train` và `X_test`.

Sau khi fit model, ta có thể trích các mean trong hai class bằng attribute `means_`. Đây là average của từng predictor trong từng class, và được LDA dùng làm estimate của μ_k . Các giá trị này gợi ý rằng return của hai ngày trước có xu hướng âm vào những ngày thị trường tăng, và return của những ngày trước có xu hướng dương vào những ngày thị trường giảm.

```
In [24]: lda.means_
```

```
Out[24]: array([[ 0.04,  0.03],
                [-0.04, -0.03]])
```

Estimated prior probability được lưu trong attribute `priors_`. Package `sklearn` thường dùng dấu `_` ở cuối như vậy để biểu thị một quantity được estimate khi dùng method `fit()`. Ta có thể chắc chắn entry nào tương ứng với label nào bằng cách xem attribute `classes_`.

```
In [25]: lda.classes_
```

```
Out[25]: array(['Down', 'Up'], dtype='<U4')
```

Output của LDA cho biết $\hat{\pi}_{\text{Down}} = 0.492$ và $\hat{\pi}_{\text{Up}} = 0.508$.

```
In [26]: lda.priors_
```

```
Out[26]: array([0.492, 0.508])
```

Các linear discriminant vector có thể được tìm thấy trong attribute `scalings_`:

```
In [27]: lda.scalings_
```

```
Out[27]: array([[ -0.642],
                [-0.513]])
```

Các giá trị này cho linear combination của `Lag1` và `Lag2` được dùng để

tạo LDA decision rule. Nói cách khác, đây là các multiplier của các phần tử của $X = x$ trong (4.24). Nếu $-0.64 \times \text{Lag1} - 0.51 \times \text{Lag2}$ lớn, LDA classifier sẽ dự đoán thị trường tăng; nếu nhỏ, LDA classifier sẽ dự đoán thị trường giảm.

```
In [28]: lda_pred = lda.predict(X_test)
```

Như ta đã thấy khi so sánh các classification method (Mục 4.5), prediction của LDA và logistic regression gần như giống hệt nhau.

```
In [29]: confusion_table(lda_pred, L_test)
```

```
Out[29]:
```

	Truth	Down	Up
Predicted			
Down		35	35
Up		76	106

Ta cũng có thể estimate probability của mỗi class cho từng point trong một training set. Áp dụng threshold 50% cho posterior probability thuộc class một cho phép ta tái tạo các prediction chứa trong `lda_pred`.

```
In [30]: lda_prob = lda.predict_proba(X_test)
np.all(
    np.where(lda_prob[:,1] >= 0.5, 'Up', 'Down') == lda_pred
)
```

```
Out[30]: True
```

Ở trên, ta dùng function `np.where()` để tạo một array có giá trị 'Up' tại các index mà column thứ hai của `lda_prob` (estimated posterior probability của 'Up') lớn hơn 0.5. Với problem có nhiều hơn hai class, label được chọn là class có posterior probability lớn nhất:

np.where()

```
In [31]: np.all(
    [lda.classes_[i] for i in np.argmax(lda_prob, 1)] ==
    lda_pred
)
```

```
Out[31]: True
```

Nếu muốn dùng posterior probability threshold khác 50% để đưa ra prediction, ta có thể làm điều đó dễ dàng. Chẳng hạn, giả sử ta chỉ muốn dự đoán thị trường giảm nếu rất chắc chắn rằng thị trường thực sự sẽ giảm trong ngày đó — ví dụ nếu posterior probability ít nhất là 90%. Ta biết column đầu tiên của `lda_prob` tương ứng với label `Down` sau khi đã kiểm tra attribute `classes_`, vì vậy ta dùng column index 0 thay vì 1 như ở trên.

```
In [32]: np.sum(lda_prob[:,0] > 0.9)
```

```
Out[32]: 0
```

Không có ngày nào trong năm 2005 đạt threshold đó! Trên thực tế,

posterior probability lớn nhất cho việc thị trường giảm trong toàn bộ năm 2005 là 52.02%.

LDA classifier ở trên là classifier đầu tiên từ thư viện **sklearn**. Ta sẽ dùng thêm một số object khác từ thư viện này. Các object tuân theo một cấu trúc chung giúp đơn giản hóa những task như cross-validation, mà ta sẽ gặp trong Chương 5. Cụ thể, các method trước hết tạo một generic classifier mà chưa tham chiếu đến dữ liệu nào. Classifier này sau đó được fit vào dữ liệu bằng method `fit()`, và prediction luôn được tạo bằng method `predict()`. Pattern gồm việc đầu tiên instantiate classifier, sau đó fit nó, rồi tạo prediction là một design choice explicit của **sklearn**. Tính uniform này giúp ta có thể cleanly copy classifier để fit trên các dữ liệu khác nhau; ví dụ các training set khác nhau phát sinh trong cross-validation. Standard pattern này cũng cho phép hình thành workflow theo cách có thể dự đoán trước.